

Ubuntu 24.04 + ROS 2 + Gazebo + PX4 复现实验方案

本文档面向当前机器：**Ubuntu 24.04 LTS + R7 4800U + 16GB 内存**。目标是放弃 ROS1 Noetic/Gazebo Classic 原样运行路线，改用 **ROS 2 Jazzy + 新 Gazebo + PX4 SITL**，参考现有 catkin_ws / damad_ws 的控制思路，重建一个能复现论文 **Interactive Real-time Leader Follower Control System for UAV and UGV** 主要算法的实验环境。

核心判断：

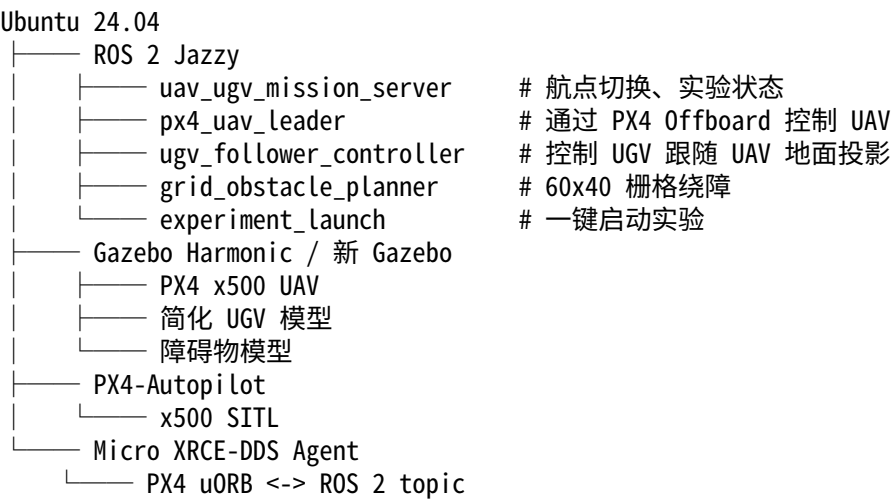
- Ubuntu 24.04 原生适合 **ROS 2 Jazzy**，不适合强行跑 ROS1 Noetic + Gazebo Classic。
- PX4 在 Ubuntu 24.04 上应走 **新 Gazebo** 路线，而不是 make px4_sitl gazebo-classic。
- 旧 catkin_ws 不能直接搬到 ROS 2，但里面的控制逻辑可以参考：iris0_cruise.py、platoon.py、spawn_2cars.launch、2cars.sh。
- 为了你的电脑性能和开发成功率，建议 **PX4 只负责 UAV**，UGV 用 ROS 2 + Gazebo 简化差速/阿克曼模型控制。这样仍然满足论文主算法目标。

参考官方文档：

- ROS 2 Jazzy 支持 Ubuntu 24.04：https://docs.ros.org/en/jazzy/Installation.html
- Gazebo Harmonic 支持 Ubuntu 24.04：https://gazebo.org/docs/harmonic/install_ubuntu/
- PX4 新 Gazebo 仿真：https://docs.px4.io/main/en/sim_gazebo_gz/
- PX4 ROS 2 通信架构：https://docs.px4.io/main/en/ros2/user_guide
- PX4 uXRCE-DDS 桥接：https://docs.px4.io/main/en/middleware/uxrce_dds
- PX4 ROS 2 Offboard 示例：https://docs.px4.io/main/en/ros2/offboard_control.html

1. 总体路线

推荐架构



为什么不建议一开始让 UGV 也用 PX4

PX4 支持 rover，但 **UAV + rover 双 PX4 实例 + ROS 2 + Gazebo** 对新手调试复杂度明显增加。论文主算法并不依赖 rover 飞控，只需要 UGV 能读取位姿、接收速度/转角命令、跟随 leader 并绕障。因此：

- **阶段 1 到 3**：PX4 控制 UAV；UGV 用 ROS 2 控制。

- **阶段 4 可选：**把 UGV 替换成 PX4 rover。

这样更符合你的硬件条件，也更容易交付实验结果。

2. 性能配置建议

你的 R7 4800U + 16GB 内存可以跑，但要避免重负载。

项目	建议
UAV 数量	1 架 PX4 x500
UGV 数量	1 辆简化 UGV
障碍物	2 到 8 个 box/cylinder
Gazebo 画质	低阴影、简单材质
RViz2	调试时开，正式跑实验可关闭
传感器频率	位姿 20 Hz，距离检测 10 Hz
不建议	YOLO、OpenPCDet、激光雷达点云融合、多 UAV 多 UGV

如果仿真卡顿：

```
HEADLESS=1 make px4_sitl gz_x500
```

或者只开 Gazebo GUI，不开 RViz2。

3. 环境配置方案

3.1 安装基础工具

```
sudo apt update
sudo apt install -y \
    git curl wget lsb-release gnupg software-properties-common \
    build-essential cmake ninja-build python3-pip python3-venv \
    python3-colcon-common-extensions python3-rosdep \
    python3-vcstool python3-argcomplete
```

初始化 rosdep：

```
sudo rosdep init
rosdep update
```

如果提示已经初始化，跳过 sudo rosdep init。

3.2 安装 ROS 2 Jazzy

Ubuntu 24.04 对应 ROS 2 Jazzy。

```
sudo apt install -y software-properties-common
sudo add-apt-repository universe
sudo apt update
sudo apt install -y curl
```

安装 ROS 2 apt 源：

```
sudo apt install -y ros2-apt-source
```

如果系统没有 ros2-apt-source，使用官方安装页中的最新版 ros2-apt-source_*.deb 方式安装。

安装桌面版和开发工具：

```
sudo apt update
sudo apt install -y ros-jazzy-desktop ros-dev-tools
```

配置环境：

```
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
source /opt/ros/jazzy/setup.bash
```

验证：

```
ros2 run demo_nodes_cpp talker
```

另开终端：

```
source /opt/ros/jazzy/setup.bash
ros2 run demo_nodes_py listener
```

3.3 安装 Gazebo Harmonic

ROS 2 Jazzy + Ubuntu 24.04 推荐使用 Gazebo Harmonic。

```
sudo apt-get update
sudo apt-get install -y curl lsb-release gnupg
sudo curl https://packages.osrfoundation.org/gazebo.gpg \
  --output /usr/share/keyrings/pkgs-osrf-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/pkgs-osrf-archive-keyring.gpg] https://packages.osrfoundation.org/gazebo/
  ubuntu-stable $(lsb_release -cs) main" \
  | sudo tee /etc/apt/sources.list.d/gazebo-stable.list > /dev/null
sudo apt-get update
sudo apt-get install -y gz-harmonic
```

验证：

```
gz sim
```

3.4 安装 ROS 2 与 Gazebo 桥接包

```
sudo apt install -y \
  ros-jazzy-ros-gz \
```

```
ros-jazzy-ros-gz-sim \  
ros-jazzy-ros-gz-bridge \  
ros-jazzy-robot-state-publisher \  
ros-jazzy-joint-state-publisher \  
ros-jazzy-xacro \  
ros-jazzy-tf2-ros \  
ros-jazzy-tf-transformations \  
ros-jazzy-rviz2
```

3.5 安装 PX4-Autopilot

建议使用 PX4 当前稳定 release 或 main。如果 main 编译失败，再切到最新稳定 release。

```
cd ~  
git clone https://github.com/PX4/PX4-Autopilot.git --recursive  
cd ~/PX4-Autopilot  
bash ./Tools/setup/ubuntu.sh
```

重启终端或重启系统后，验证新 Gazebo 仿真：

```
cd ~/PX4-Autopilot  
make px4_sitl gz_x500
```

如果能打开 Gazebo 并生成 x500 UAV，PX4 基础仿真可用。

3.6 安装 Micro XRCE-DDS Agent

PX4 和 ROS 2 通信需要 Micro XRCE-DDS Agent。官方示例使用源码安装：

```
cd ~  
git clone -b v2.4.3 https://github.com/eProsima/Micro-XRCE-DDS-Agent.git  
cd Micro-XRCE-DDS-Agent  
mkdir build  
cd build  
cmake ..  
make -j$(nproc)  
sudo make install  
sudo ldconfig /usr/local/lib/
```

启动 Agent：

```
MicroXRCEAgent udp4 -p 8888
```

3.7 创建 ROS 2 工作空间

```
mkdir -p ~/uav_ugv_ros2_ws/src  
cd ~/uav_ugv_ros2_ws/src  
git clone https://github.com/PX4/px4_msgs.git  
git clone https://github.com/PX4/px4_ros_com.git
```

如果 PX4 使用 release 分支，px4_msgs 最好切到匹配分支，避免消息定义不一致。

编译：

```
cd ~/uav_ugv_ros2_ws
source /opt/ros/jazzy/setup.bash
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install
source install/setup.bash
```

4. PX4 + ROS 2 基础通信验证

4.1 终端启动顺序

终端 1：启动 Agent。

```
MicroXRCEAgent udp4 -p 8888
```

终端 2：启动 PX4 x500。

```
cd ~/PX4-Autopilot
make px4_sitl gz_x500
```

终端 3：查看 ROS 2 话题。

```
source /opt/ros/jazzy/setup.bash
source ~/uav_ugv_ros2_ws/install/setup.bash
ros2 topic list | grep fmu
```

应能看到类似：

```
/fmu/out/vehicle_odometry
/fmu/out/vehicle_status
/fmu/in/vehicle_command
/fmu/in/offboard_control_mode
/fmu/in/trajectory_setpoint
```

4.2 跑 PX4 ROS 2 Offboard 示例

```
cd ~/uav_ugv_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 run px4_ros_com offboard_control
```

成功现象：

- UAV 解锁。
- 进入 Offboard。
- 起飞并上升到示例设定高度。

注意：PX4 Offboard 控制要求持续发送 OffboardControlMode 等消息。官方文档提醒，如果低于约 2 Hz，PX4 会退出 Offboard。实际建议用 20 Hz。

5. “跑通 catkin_ws 同样效果” 的 ROS 2 方案

这里的“同样效果”不是原样运行 catkin_ws/*.sh，而是在 ROS 2 中实现相同实验能力：

原 ROS1/catkin_ws 内容	ROS 2 新实现
uav_cruise.sh	启动 PX4 x500 + ROS 2 leader 节点，让 UAV 起飞并按航点飞
2cars.sh	启动 Gazebo 世界 + UAV + UGV + follower 节点 + mission server
spawn_2cars.launch	ROS 2 launch + SDF/URDF 生成简化 UGV 和障碍物
iris0_cruise.py	改写为 px4_uav_leader，发布 PX4 Offboard setpoint
platoon.py	改写为 ugv_follower_controller，控制 UGV 跟随 UAV 投影
traffic_light.py	非论文核心，先不做；后续可选
yolo_detect.sh、lidar_cam.sh、radar_cam.sh	非论文核心，先不做

5.1 新工作空间建议结构

在 ~/uav_ugv_ros2_ws/src 下新增一个包：

```
cd ~/uav_ugv_ros2_ws/src
ros2 pkg create uav_ugv_repro --build-type ament_python \
  --dependencies rclpy geometry_msgs nav_msgs std_msgs sensor_msgs px4_msgs tf_transformations
```

建议目录：

```
uav_ugv_repro/
├── launch/
│   ├── sim_world.launch.py
│   ├── px4_uav_only.launch.py
│   └── uav_ugv_experiment.launch.py
├── models/
│   ├── simple_ugv/
│   └── obstacle_box/
├── worlds/
│   └── leader_follower_world.sdf
├── uav_ugv_repro/
│   ├── px4_uav_leader.py
│   ├── ugv_follower_controller.py
│   ├── mission_server.py
│   ├── grid_obstacle_planner.py
│   ├── gazebo_obstacle_detector.py
│   └── trajectory_logger.py
└── config/
    └── experiment.yaml
```

5.2 先实现不带论文绕障的最小实验

目标：相当于旧 uav_cruise.sh + 2cars.sh 的核心效果。

步骤：

1. Gazebo 中有一个世界。
2. PX4 x500 UAV 起飞并按航点飞。
3. Gazebo 中有一个简化 UGV。
4. ROS 2 节点读取 UAV 位姿和 UGV 位姿。
5. UGV 跟随 UAV 的地面投影。

建议先不要让 Gazebo 同时加载复杂 site_model。用一个空世界 + 简单障碍物更稳定。

5.3 UGV 实现方式

优先使用简单差速 UGV，而不是复杂阿克曼车。

控制话题：

/ugv/cmd_vel geometry_msgs/Twist

位姿来源二选一：

1. 通过 Gazebo bridge 读取 /model/ugv/pose。
2. 自己写 ugv_state_publisher，根据 /ugv/cmd_vel 积分得到 /ugv/odom。这更轻，但不是物理仿真。

推荐第一阶段用方案 2，快；第二阶段再换 Gazebo bridge。

5.4 一键实验启动目标

最终希望做到：

```
cd ~/uav_ugv_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch uav_ugv_repro uav_ugv_experiment.launch.py
```

该 launch 应启动：

- Micro XRCE-DDS Agent，或提示用户单独启动。
- Gazebo world。
- PX4 x500。
- mission_server.py。
- px4_uav_leader.py。
- ugv_follower_controller.py。
- grid_obstacle_planner.py。
- trajectory_logger.py。

如果 launch 中直接启动 PX4 比较麻烦，可以先手动三终端启动，等稳定后再整合。

6. 论文复现方案

6.1 对应关系

论文模块	新实现
QDrone / leader	PX4 x500 SITL
QBot / follower	ROS 2 + Gazebo 简化 UGV
OptiTrack	Gazebo truth pose / ROS 2 odom
Simulink Mission Server	mission_server.py
QDrone Commander	px4_uav_leader.py
QBot Commander	ugv_follower_controller.py
HC-SR04 双超声	Gazebo 障碍物真值检测或两个 ray 传感器
60x40 grid	grid_obstacle_planner.py

6.2 Leader UAV 节点

节点名：

px4_uav_leader

订阅：

/fmu/out/vehicle_odometry
/mission/current_waypoint
/mission/state

发布：

/fmu/in/offboard_control_mode
/fmu/in/trajectory_setpoint
/fmu/in/vehicle_command
/leader/pose_enu

逻辑：

1. 持续发布 OffboardControlMode。
2. 发布初始 hover setpoint。
3. 切 Offboard。
4. Arm。
5. 起飞到 1.5 m。
6. 接收 mission_server 给的航点。
7. 发布 TrajectorySetpoint。
8. 最后返回 home 并 land。

坐标注意：

- ROS 常用 ENU：x 前/东, y 左/北, z 上。
- PX4 常用 NED：x 北, y 东, z 下。
- 不要直接把 ROS ENU 的 z=1.5 塞给 PX4 NED；要在节点里做坐标转换。

6.3 Mission Server 节点

节点名:

mission_server

订阅:

/leader/pose_enu

/ugv/odom

发布:

/mission/current_waypoint

/mission/state

/mission/finished

核心规则:

```
if UAV has taken off:  
    publish waypoint[i]
```

```
if distance_xy(UAV, UGV) < 0.25:  
    i += 1
```

```
if i == len(waypoints):  
    publish home waypoint  
    request landing
```

推荐航点:

```
waypoints:  
- [0.0, 0.0, 1.5]  
- [1.5, 0.0, 1.5]  
- [1.5, 1.2, 1.5]  
- [0.0, 1.2, 1.5]  
- [0.0, 0.0, 1.5]
```

6.4 UGV Follower 节点

节点名:

ugv_follower_controller

订阅:

/leader/pose_enu

/ugv/odom

/ugv/local_goal

发布:

```
/ugv/cmd_vel
```

无障碍控制律：

```
target = UAV ground projection: [uav_x, uav_y]
dx = target_x - ugv_x
dy = target_y - ugv_y
target_yaw = atan2(dy, dx)
yaw_error = wrap_to_pi(target_yaw - ugv_yaw)
distance = sqrt(dx^2 + dy^2)
```

```
linear.x = clamp(kv * distance, 0.0, 0.5)
angular.z = clamp(kw * yaw_error, -1.0, 1.0)
```

```
if distance < 0.25:
    linear.x = 0
    angular.z = 0
```

建议参数：

```
kv: 0.4
kw: 1.2
max_speed: 0.5
max_yaw_rate: 1.0
stop_distance: 0.25
```

6.5 障碍物检测

先用简单可靠的方式，不要一开始加传感器插件。

第一版：

```
gazebo_obstacle_detector.py
```

功能：

- 读取 UGV 位姿。
- 读取预设障碍物位置。
- 判断 UGV 前方 0.30 m、左右两侧是否有障碍。
- 输出：

```
/ugv/front_left_obstacle  std_msgs/Bool
/ugv/front_right_obstacle std_msgs/Bool
```

第二版再换成 Gazebo ray 传感器。

6.6 网格绕障

节点名：

```
grid_obstacle_planner
```

地图：

```
width: 60
height: 40
resolution: 0.1
origin: [-3.0, -2.0]
inflation_radius_cells: 2
```

逻辑：

1. 无障碍时，/ugv/local_goal = UAV ground projection。
2. 左传感器触发时，在 UGV 前方 0.3 m 偏左位置标记 3 格障碍。
3. 右传感器触发时，在 UGV 前方 0.3 m 偏右位置标记 3 格障碍。
4. 对障碍膨胀 2 格。
5. 从 UGV 当前格到 UAV 投影格跑 A* 或 BFS。
6. 取路径前方第 3 到第 6 个点作为局部目标。

这比论文原始算法更稳，但仍然复现了论文主要思想：**边走边记录障碍，基于栅格重新生成 follower 路径。**

6.7 轨迹记录与实验结果

节点名：

trajectory_logger

记录：

time, uav_x, uav_y, uav_z, ugv_x, ugv_y, ugv_yaw, waypoint_id, obstacle_left, obstacle_right

保存：

~/uav_ugv_ros2_ws/results/run_001.csv

实验图：

- UAV XY 轨迹。
- UGV XY 轨迹。
- 障碍物位置。
- UGV 与 UAV 的距离随时间变化。
- 航点切换时刻。

7. 分阶段执行计划

阶段 A：PX4 单机跑通

验收：

```
make px4_sitl gz_x500
```

能看到 UAV。

阶段 B：ROS 2 能读写 PX4

验收：

```
MicroXRCEAgent udp4 -p 8888
ros2 topic list | grep fmu
ros2 run px4_ros_com offboard_control
```

UAV 能起飞。

阶段 C：简化 UGV 跑通

验收：

```
ros2 topic pub /ugv/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.2}, angular: {z: 0.0}}"
```

UGV 能前进，/ugv/odom 有输出。

阶段 D：复现旧 uav_cruise.sh 效果

- 验收：
- UAV 起飞。
 - UAV 依次飞到 3 到 5 个航点。
 - 保存 UAV 轨迹。

阶段 E：复现旧 2cars.sh 的核心效果

- 验收：
- Gazebo 中同时有 UAV 和 UGV。
 - UGV 跟随 UAV 地面投影。
 - UGV 接近 UAV 后 Mission Server 切换下一个航点。

阶段 F：复现论文绕障

- 验收：
- 放置两个障碍物。
 - UGV 检测障碍后栅格地图出现占据格。
 - UGV 绕过障碍继续靠近 UAV。
 - 输出轨迹图和 CSV。

8. 与当前旧代码的参考关系

旧文件	借鉴内容	注意
catkin_ws/src/site_model/src/iris0_cruise.py	起飞、定高、航点巡航、速度平滑	ROS1/MAVROS 写法不能直接用，改成 px4_msgs

旧文件	借鉴内容	注意
catkin_ws/src/site_model/src/platoon.py	follower 朝目标点转向、速度随距离变化	旧代码是车跟车，新代码改成车跟 UAV 投影
catkin_ws/src/site_model/src/iris0_follow.py	UAV/UGV 位姿差控制思想	方向相反，旧代码是 UAV 跟 UGV
catkin_ws/2cars.sh	多节点启动顺序	改成 ROS 2 launch
catkin_ws/src/site_model/worlds/spawn_cars.world	障碍物和场景布置思路	SDF 可参考，但不要直接依赖旧 Gazebo Classic 插件
damad_ws/src/control_pkg/src/follow.py	简单 follower 控制	ROS1 topic 名称和 ackermann 接口需重写

9. 风险和替代方案

风险 1：PX4 + ROS 2 Jazzy 示例编译不顺

处理：

- 先只跑 make px4_sitl gz_x500。
- 再确认 MicroXRCEAgent。
- px4_msgs 与 PX4 分支保持匹配。
- 如果 px4_ros_com 对 Jazzy 有编译问题，可以不依赖它的示例，自己只用 px4_msgs 写 Python/C++ 节点。

风险 2：Gazebo + PX4 太卡

处理：

- 用 HEADLESS=1 make px4_sitl gz_x500。
- 不开 RViz2。
- UGV 不用复杂模型。
- 传感器频率降低到 10 Hz。

风险 3：PX4 Offboard 控制坐标系混乱

处理：

- 第一版固定 UAV 只飞简单矩形航点。
- 在日志中同时打印 ENU 和 NED。
- z 方向尤其注意：ROS ENU 是 z 向上，PX4 NED 是 z 向下。

风险 4：新 Gazebo 插件资料少

处理：

- 先不用 ray 传感器，用障碍物真值检测。
- 论文复现重点是算法，不是传感器插件。

10. 最推荐的实际执行顺序

1. 安装 ROS 2 Jazzy。

2. 安装 Gazebo Harmonic。
3. 安装 PX4-Autopilot。
4. 跑通 `make px4_sitl gz_x500`。
5. 安装并启动 Micro XRCE-DDS Agent。
6. 跑通 `ros2 topic list | grep fmu`。
7. 跑通 PX4 Offboard 示例。
8. 新建 `uav_ugv_repro` ROS 2 包。
9. 先做 `px4_uav_leader.py`，实现 UAV 航点飞行。
10. 再做简化 UGV，能 `/ugv/cmd_vel` 控制。
11. 做 `mission_server.py`，实现“UGV 接近后切换航点”。
12. 做 `grid_obstacle_planner.py`，实现论文栅格绕障。
13. 记录轨迹并画图。

11. 最小交付标准

实验最终至少应能展示：

1. Gazebo 中 UAV 起飞并按航点飞行。
2. UGV 跟随 UAV 地面投影。
3. Mission Server 控制航点切换。
4. UGV 遇障后更新 60x40 栅格地图。
5. UGV 绕过障碍后继续跟随 UAV。
6. 有 UAV/UGV 轨迹图、障碍物位置图、距离曲线。

这已经覆盖论文的主要算法流程，不需要复现 QUANSER、OptiTrack、Simulink 和 HC-SR04 硬件细节。